

IBM Data Engineering Course | Coursera

ETL and Data Pipelines with Shell, Airflow and Kafka

Summary

Module one

Data Processing Techniques

ETL and ELT Processes

Lesson 1: ETL Fundamentals

Objectives

By the end of this lesson, you will be able to:

- Understand what ETL means and its significance.
 - Explain the three core stages: **Extraction**, **Transformation**, and **Loading**.
 - Recognize real-world **use cases** for ETL pipelines.
-

What is ETL?

ETL = Extract, Transform, Load

It's an automated **data pipeline engineering methodology** that:

- Acquires raw data from various sources
 - Prepares and standardizes it (transformation)
 - Loads it into a new environment like a **data warehouse** or **data mart** for analytics
-

1. Data Extraction

Extraction is the process of retrieving data from one or more sources.

Common techniques:

- **Web Scraping:** Parsing HTML using tools like Python or R
- **APIs:** Querying structured data via programmatic endpoints
- **Batch Extraction:** From static sources like archives

- **Streaming Extraction:** From dynamic sources like weather feeds, social media, IoT devices

 *Goal:* Make the data **accessible** to your application.

2. Data Transformation

Also called **Data Wrangling**. This stage **cleans and reshapes** the data for analysis.

Common tasks:

- **Cleaning:** Fixing errors, filling missing values
- **Filtering:** Selecting only relevant records
- **Joining:** Merging multiple sources
- **Feature Engineering:** Creating KPIs or new metrics
- **Formatting & Typing:** Making the data compatible with the target system

 *Goal:* Make the data **useful** and **consistent**.

3. Data Loading

This step writes the transformed data into its destination system.

Destinations include:

- Databases
- Data Warehouses
- Data Marts

 *Goal:* Make the data **available for analytics** and **decision-making**.

Use Cases of ETL

- **Digitizing analog data:** From paper, audio tapes, images, etc.
 - **Historical data capture:** Extracting from OLTP for use in OLAP systems
 - **Dashboard KPIs:** ETL prepares features for dashboards in ops, sales, and marketing
 - **ML & AI:** Prepares training data for models used in forecasting and classification
-

Key Takeaways

- **ETL = Extract + Transform + Load**
 - It's used to prepare data from multiple sources for analysis
 - Extraction = Get the data 
 - Transformation = Make it clean & usable 
 - Loading = Move it to a usable system 
 - ETL enables analytics, dashboards, machine learning, and more.
-

 *Video Highlight:* ETL is essential for **curating data and making it accessible** for insights and machine learning-based decision-making.

Lesson 2: ELT Basics

Objectives

By the end of this lesson, you will be able to:

- Define what ELT (Extract, Load, Transform) means
 - Understand its **use cases** and **advantages**
 - Explain why ELT is an **emerging trend** in modern data engineering
-

What is ELT?

ELT = Extract, Load, Transform

It's a modern variation of ETL, where:

- Data is first **extracted** from various sources
- Then **loaded directly as raw data** into the destination (e.g., a data lake)
- Finally, **transformation** happens *on-demand* within that environment

 *Core difference:* In ELT, data is transformed **after** loading, not before.

1. Data Extraction

Just like ETL:

- Extracts data from various sources

- Often happens asynchronously
 - Reads the data into the pipeline-ready system
-

2. Data Loading

- The raw data is **loaded as-is** into its final environment
 - This environment is often a **cloud-based** analytics platform (like a data lake)
 - No pre-formatting or cleaning is required before loading
-

3. Data Transformation

- Transformation happens **after loading**, and is:
 - **On-demand**
 - **Dynamic**
 - **Interactive**
- Leverages modern tools for:
 - Visualization
 - Machine Learning
 - Advanced Analytics

 *Key Benefit:* Users can explore and manipulate data **freely**, with no early assumptions.

Use Cases for ELT

ELT shines in:

- **Big Data & High Performance Computing**
 - **Real-time analytics** on streaming sources
 - **Globally distributed data systems**
 - **Flexible data product development** from the same raw data
 - **Cloud-native workflows** where cost and scalability matter
-

Why ELT Is an Emerging Trend

-  **Cloud computing** is exploding in power and affordability
-  ELT fits naturally into cloud platforms (data lakes, BigQuery, Snowflake, etc.)
-  Transforms can be done *only when needed*, reducing:
 - Information loss 

- Pipeline rigidity 🧱
- 🔄 Allows reuse of raw data for **multiple pipelines**
- 💰 Pay only for what you use (no underutilized hardware)

📄 Key Takeaways

- **ELT = Extract → Load → Transform:** Raw data first, transform later.
- It's ideal when you want:
 - **Flexibility**
 - **Scalability**
 - **Speed**
- Powered by modern **cloud platforms** that separate storage from compute.
- Helps avoid **early data loss** by keeping the raw source intact.

🎥 *Video Highlight:* ELT gives you the power to explore and transform raw data *your way, on demand* — unlocking limitless analytics potential in the cloud ☁️ ⚡

🧠 Lesson 3: ELT and ETL

🎯 Objectives

After this lesson, you will be able to:

- List the **key differences** between ETL and ELT
- Understand ELT as an **evolution** of ETL
- Describe the **shift from ETL to ELT** in modern data workflows

🔄 ETL vs. ELT: Key Differences

Feature	ETL	ELT
Transformation	Before loading (in pipeline)	After loading (in destination)
Flexibility	Rigid, serves a specific function	Highly flexible, supports ad-hoc analysis
Data Types	Mostly structured, relational data	Any data (structured, semi/unstructured)
Scalability	Limited (on-premises)	Scalable (cloud-native)
Time-to-Insight	Slower, depends on dev teams	Faster, self-serve dashboards/models

Feature	ETL	ELT
User Access	Dev-centric, limited access	Open, supports wider user base

Why ELT is the Evolution of ETL

- ETL uses a **staging area** to hold raw data temporarily
 - Similar in spirit to modern **data lakes**
 - But these are often siloed and internal
- ELT transforms data *after* loading it into **shared cloud repositories** like data lakes
 - Promotes **data democratization**
 - Supports **enterprise-wide access**

 *Takeaway:* The traditional **staging area** has evolved into the **data lake**, a central piece in ELT architecture.

The Shift to ELT: What's Driving It?

-  Demand for **real-time insights** & faster decision-making
 -  Rise of **cloud computing** and **on-demand scalability**
 -  Growth of **self-service analytics** for non-technical users
 -  Reduction in **time-to-insight** compared to slow ETL workflows
 -  Breaks down **data silos** that restrict access
-

Does ETL Still Have a Role?

Yes!

- ETL is still useful for:
 - Pre-defined **data warehouse** pipelines
 - Environments where **data quality checks** must be applied early
 - But ELT is more **adaptive** to modern needs
-

Key Takeaways

- Main differences: **Where transformation happens**, flexibility, data types supported, and time-to-insight
- ELT = evolution of ETL, enabled by **cloud infrastructure** and **modern analytics tools**

- ETL isn't obsolete, but ELT is now the **preferred model** for agile, scalable data pipelines

 *Video Highlight:* The future of data engineering is moving from **rigid pipelines** to **flexible platforms** — and that's the power of ELT  

Lesson 4: Data Extraction Techniques

Objectives

After this lesson, you will be able to:

- List examples of **raw data sources**
- Describe common **data extraction techniques**
- Relate **use cases** to **data sources** and **extraction methods**

Examples of Raw Data Sources

-  Archived text and images from **paper documents** and **PDFs**
-  **Web pages** (text, tables, images, links)
-  **Analog audio** and **video** recordings (e.g., magnetic tapes, real-time streams)
-  **Survey, statistical, economic, and transactional** data (e.g., business, financial, real estate, POS transactions)
-  **Event-based data** (e.g., social media streams, weather networks, IoT sensor streams)
-  **Medical records** (prescriptions, treatments, medical images)
-  **Genetic data** (DNA and RNA samples)

 *Note:* Many of these sources are **sensitive** and require strong **privacy** protections.

Common Data Extraction Techniques

Technique	Purpose
OCR (Optical Character Recognition)	Digitizes scanned text from documents
ADC (Analog-to-Digital Converters)	Converts analog audio into digital formats

Technique	Purpose
CCD (Charge-Coupled Devices)	Captures and digitizes images
Polling and Census Methods	Collects survey and statistical data
Cookies & User Logs	Tracks human and system behavior
Web Scraping	Extracts content (text, images, tables, links) from websites
APIs	Interfaces to retrieve data from online platforms and repositories
SQL & NoSQL Queries	Extracts structured and semi-structured data from databases
Edge Computing Devices	Processes raw data locally to extract key features
Biomedical Devices	Captures medical and genetic data (e.g., DNA sequences)

Use Cases: Matching Sources and Techniques

Use Case	Source	Extraction Technique
Central Repository Integration	Multiple structured data sources	APIs
Historical Event Capture	Asynchronous event streams	APIs
Reducing IoT Data Volume	IoT devices	Edge computing
Medical Diagnostics	Medical imaging and sensors	Biomedical devices

 *Insight:* Sometimes, **full raw data** must still be stored first, and processed **later**, when immediate extraction isn't practical.

Key Takeaways

- Raw data comes from a **wide variety** of sources, many of them sensitive
- Extraction techniques depend on **data type** and **intended use**
- **APIs, Web Scraping, SQL, NoSQL** are core tools in modern data extraction
- **Edge computing** and **biomedical devices** are examples of specialized extraction at the data source

 *Video Highlight:* Data extraction is the **first step** towards turning vast oceans of raw information into **actionable insights**  

Lesson 5: Introduction to Data Transformation Techniques

Objectives

After this lesson, you will be able to:

- Name common **data transformation techniques**
 - Compare **schema-on-write** vs **schema-on-read** approaches
 - List ways **information can be lost** during transformation
-

What is Data Transformation?

Data transformation involves **formatting data** to meet the needs of an application, typically by:

- Changing **data types** (e.g., integer, float, string, category)
 - **Structuring** data (e.g., converting JSON/XML/CSV to database tables)
 - **Anonymizing** or **encrypting** data for privacy and security
 - **Cleaning** data (e.g., removing duplicates, filling missing values)
 - **Normalizing** units (e.g., converting all prices to a common currency)
 - **Filtering, sorting, aggregating, and binning** data
 - **Joining** or **merging** data from different sources
-

Schema-on-Write vs Schema-on-Read

Feature	Schema-on-Write	Schema-on-Read
Process	Apply schema before storage	Apply schema after reading raw data
Used in	Traditional ETL pipelines	Modern ELT pipelines
Pros	Structured, fast queries, stable	Flexible, multiple schema views possible
Cons	Limits data versatility	May involve slower queries
Data loss	Possible if schema is strict	Original raw data is preserved

Ways Information Can Be Lost During Transformation

Method	Example
Lossy Compression	Converting floating points to integers, reducing bit rates in audio/video
Filtering	Permanently removing subsets of data

Method	Example
Aggregation	Losing detail when summarizing (e.g., yearly vs daily sales data)
Edge Computing Devices	Discarding raw data, relying only on extracted features (e.g., false negatives in surveillance systems)

 *Visualization:*

- **Raw Data** = large volume, full information + noise
- **Transformed Data** = smaller volume, risk of information loss

Key Takeaways

- Data transformation adjusts data to **fit application needs**
- Techniques include **typing, structuring, cleaning, normalizing, aggregating, and merging**
- **Schema-on-write** = strict, fast queries but less flexible
- **Schema-on-read** = flexible, preserves raw data
- Information loss can happen via **compression, filtering, aggregation, and edge processing**

 *Video Highlight:* Transformation is powerful, but **care must be taken** not to lose valuable information along the way! 🌱 ✨

Lesson 6: Data Loading Techniques

Objectives

After this lesson, you will be able to:

- List common **data loading strategies and techniques**
 - Differentiate **batch loading** from **stream loading**
 - Explain **push** vs **pull** data loading methodologies
 - Understand different **data loading plans**
 - Describe **parallel loading** for efficiency
-

Data Loading Strategies

Strategy	Description	When to Use
Full Loading	Load all data into an empty warehouse	New system setup, initial history load
Incremental Loading	Load only new or changed data since last load	Ongoing operations, accumulating history

- **Full Loading** = No existing content
- **Incremental Loading** = Appends data without overwriting

Stream Loading vs Batch Loading

Type	Description	Examples
Stream Loading	Continuous updates as new data arrives	IoT sensors, social media feeds, real-time events
Batch Loading	Periodic updates scheduled at intervals	Daily stock updates, scheduled tasks (Cron jobs, Windows Task Scheduler)

Push vs Pull Methodologies

Method	How it Works	Best For
Push	Source system sends data to the destination	Real-time streaming, active systems
Pull	Destination requests data from the source	Scheduled batch transactions

- **Push** = Source-initiated
- **Pull** = Destination-initiated

Parallel vs Serial Loading

Type	Description	Advantage
Serial Loading	Data loaded one after another sequentially	Simple but slower
Parallel Loading	Multiple data streams or chunks loaded simultaneously	Much faster and efficient for big data

◆ *Parallel loading* improves performance by:

- Loading from multiple sources at once
- Splitting large files into chunks and loading them simultaneously

Key Takeaways

- **Full loading** = All data at once; **Incremental loading** = Only changes
- **Stream loading** = Continuous; **Batch loading** = Scheduled intervals
- **Push** = Data pushed by source; **Pull** = Data pulled by destination
- **Parallel loading** accelerates large data loading compared to **serial loading**

 *Video Highlight:* Efficient loading strategies = Better performance and fresher data! 🚀 ⭐

Lesson's Summary:

- ETL stands for Extract, Transform, and Load
- Loading means writing the data to its destination environment
- Cloud platforms are enabling ELT to become an emerging trend
- The key differences between ETL and ELT include the place of transformation, flexibility, Big Data support, and time-to-insight
- There is an increasing demand for access to raw data that drives the evolution from ETL, which is still used, to ELT, which enables ad-hoc, self-serve analytics
- Data extraction often involves advanced technology including database querying, web scraping, and APIs
- Data transformation, such as typing, structuring, normalizing, aggregating, and cleaning, is about formatting data to suit the application
- Information can be lost in transformation processes through filtering and aggregation
- Data loading techniques include scheduled, on-demand, and incremental
- Data can be loaded in batches or streamed continuously

Can you differentiate between ETL and ELT?

Drag and drop the below terms into the correct bins

ETL		ELT
Source and destination databases are different	Submit Score 100 % Start Over	Unstructured Data
Data size is small		Source and destination databases are same
Structured Data		Data size is huge
Complex data transformations		Simple data transformations

Module Two

Etl & Data Pipelines: Tools and Techniques

ETL using Shell Script

ETL Techniques

Objectives

- Understand the ETL process: **Extract, Transform, Load**
- Learn about **staging areas**, **ETL pipelines**, and **workflow orchestration**
- Review popular **ETL tools** and their features

1. What is ETL?

ETL (Extract, Transform, Load) is a process for:

- **Extracting** data from multiple, often disparate sources
- **Transforming** the data into a usable format
- **Loading** the data into a final destination like a database, data warehouse, or data lake

 **ETL forms the backbone of data pipelines** that feed analytic systems.

2. ETL Stages

Extract

- **Collect** data from sources like:
 - IoT sensors
 - Social media feeds
 - Enterprise databases
 - Unstructured files (emails, scanned documents)

Transform

Performed in a **staging area** to ensure data quality and compatibility:

- Cleaning (fix errors, fill missing values)
- Filtering (select required fields)
- Joining (combine multiple sources)
- Normalizing (common units and formats)
- Structuring (format changes: JSON → Tables)
- Feature Engineering (create KPIs, machine learning features)
- Anonymizing and Encrypting (privacy/security)
- Sorting and Aggregating

Load

- Move transformed data into the target system:
 - Flat files (CSV, Excel)
 - Databases
 - Data warehouses/marts/lakes
 - Must satisfy **schema integrity constraints** (e.g., uniqueness, foreign keys)
-

3. ETL Workflows as Pipelines

- **Data pipelines** feed data in **packets** to maximize throughput.
- ETL processes (Extract → Transform → Load) work **simultaneously on different packets**.
- Often uses **batch processing** at regular intervals or triggered by events.

 Example: Moving daily transactions from OLTP to OLAP systems.

4. Staging Areas

- Temporary areas where:
 - Data is checked for changes

- Updates are applied
 - Data is transformed before loading
 - Helpful when integrating siloed systems across the enterprise.
-

5. ETL Workflows as DAGs

- Complex ETL pipelines can be organized using **Directed Acyclic Graphs (DAGs)**.
 - **Apache Airflow** is a popular orchestration tool:
 - Tasks represented as nodes
 - Dependencies shown as arrows
 - Supports parallel and scheduled execution
-

6. Popular ETL Tools

Tool	Key Features
Talend Open Studio	Open-source, drag-and-drop, supports Big Data, generates Java code
AWS Glue	Serverless ETL service, schema suggestions, job automation
IBM InfoSphere DataStage	Scalable platform, drag-and-drop interface, parallel processing
Alteryx	No-code ETL creation, self-service analytics platform
Apache Airflow	Open-source, code-based workflow management, highly scalable
Pandas (Python library)	Open-source, great for prototyping and exploration (not Big Data-scalable)

Key Takeaways

- ETL is fundamental for **preparing, cleaning, and integrating** data.
 - **Staging areas** and **data pipelines** help manage complex data flows.
 - Workflow orchestration (e.g., DAGs with Airflow) improves **efficiency and scalability**.
 - Multiple **ETL tools** exist, each fitting different needs from prototyping (Pandas) to production (AWS Glue, Talend).
-

ETL Using Shell Scripting

Objectives:

- Understand how **Shell Scripting** can be used to build an ETL pipeline.
 - Learn how to **schedule ETL scripts** using **cron jobs**.
-

Scenario:

Task: Report the **hourly** average, minimum, and maximum temperatures from a remote sensor and **feed** the results to a dashboard **every minute**.

You are provided with:

- **get_temp_api** → reads and prints the current temperature.
 - **load_stats_api** → loads computed statistics to a repository.
-

ETL Pipeline Workflow:

1. Extraction (E)

- Use `get_temp_api` to **read** the temperature.
- **Append** the reading to a log file `temperature.log`.
- Keep **only the latest 60 readings** (1 hour of data).
- `tail -n 60 temperature.log > temp && mv temp temperature.log`

2. Transformation (T)

- Create a **Python script** `get_stats.py` that:
 - Reads `temperature.log`
 - Calculates **min**, **max**, and **average** temperatures.
 - Outputs results to `temp_stats.csv`.
- `python3 get_stats.py temperature.log temp_stats.csv`

3. Loading (L)

- Use `load_stats_api` to **upload** the `temp_stats.csv` to the dashboard system:
 - `load_stats_api temp_stats.csv`
-

Writing the Shell Script (Temperature_ETL.sh)


```
#!/bin/bash

# Extract: Read temp and append
get_temp_api >> temperature.log

# Keep only last 60 readings
tail -n 60 temperature.log > temp && mv temp temperature.log

# Transform: Calculate stats
python3 get_stats.py temperature.log temp_stats.csv

# Load: Upload stats
load_stats_api temp_stats.csv
```

✅ After writing, make it **executable**:

```
chmod +x Temperature_ETL.sh
```

Scheduling with Cron:

- Open crontab editor:
 - `crontab -e`
 - Add the cron job to run **every minute**:
 - `* * * * * /path/to/Temperature_ETL.sh`
 - Save and exit. Your ETL job is now **running in production!** 🚀
-

Key Takeaways:

- **Shell scripts** can automate full ETL pipelines simply.
 - **cron** allows scheduling jobs **at any frequency**.
 - Keeping **buffered logs** ensures **fresh** and **relevant** data is processed.
 - Integration between Bash and **Python** is seamless and powerful.
-

Lesson's Summary:

- ETL pipelines are created with Bash scripts
 - ETL jobs can be run on a schedule using cron
-

An Introduction to Data Pipelines:

Introduction to Data Pipelines

Objectives:

- Define what a **pipeline** is.
 - Understand **latency** and **throughput** in data pipelines.
 - Explore common **use cases** for data pipelines.
-

What is a Pipeline?

- A **pipeline** is a set of processes **connected sequentially**.
 - **Output** from one process = **Input** for the next process.
 - Example:
 - Chain of friends passing boxes → each friend processes a box: **get** → **pass**.
 - **Mass production** works similarly, though transformations may vary between stages.
-

What is a Data Pipeline?

- A **Data Pipeline** is a system that **moves or modifies data**.
 - It:
 - **Extracts** data
 - Optionally **transforms** data
 - **Loads** data to its final destination.
 - Focus here: **Software-driven** architectures (commands, programs, threads).
 - In **Linux**, even simple bash pipes (|) connect small processes into pipelines.
 - Data flows as **packets**:
 - Could be single records, events, or big batches.
-

Data Pipeline Performance:

1. Latency

- **Definition:** Total time for a single packet to move through the pipeline.
- **Latency** = Sum of times spent at each processing stage.
- **Bottleneck:** Slowest stage determines overall latency.
- **Example:** Even with fast internet, slow server = slow web page load.

2. Throughput

- **Definition:** Amount of data moving through the pipeline **per unit of time**.
 - **Higher throughput:** Process larger packets or more packets in the same time.
 - **Example:** Bigger boxes passed by friends = higher productivity.
-



Common Use Cases for Data Pipelines:

- **File backups** (simple copy/move with no transformation).
 - **Data lake integration** (gathering raw data from various sources).
 - **Data warehouse loading** (moving transaction records for analytics).
 - **IoT device streaming** (feeding dashboards or AI systems).
 - **Preparing data for ML** (cleaning, transforming for model training).
 - **Messaging systems** (email, SMS, video conferencing like Zoom or Teams).
-



Key Takeaways:

- A **data pipeline** moves or transforms data between stages.
 - **Latency** and **throughput** are critical for design and performance.
 - Pipelines can serve **very simple** to **very complex** needs — from backup tasks to powering **real-time** video meetings.
-

Key Data Pipeline Processes



Objectives:

- List **key stages** in a data pipeline.
 - Understand **monitoring** considerations.
 - Learn **solutions** for mitigating **bottlenecks** in data flow.
-



Key Stages in a Data Pipeline:

1. **Extraction**
 - Pulling data from one or more sources.
2. **Ingestion**
 - Feeding the extracted data into the pipeline.
3. **Transformation (optional)**
 - Modifying/processing data inside the pipeline.

4. **Loading**
 - Storing the final data into a destination (database, data warehouse, etc.).
 5. **Scheduling/Triggering**
 - Automating when and how pipeline jobs run.
 6. **Monitoring**
 - Keeping an eye on pipeline health and performance.
 7. **Maintenance and Optimization**
 - Ensuring continuous, efficient operation of the pipeline.
-

Data Pipeline Monitoring Considerations:

- **Latency:**
 - Time a packet takes to flow through the pipeline.
 - **Throughput Demand:**
 - Volume of data passing through over time.
 - **Errors and Failures:**
 - Issues from network problems or source/destination system failures.
 - **Utilization Rate:**
 - How fully the system's resources are being used (affects costs).
 - **Logging and Alerts:**
 - Systems must log events and alert administrators immediately upon failures.
-

Solving Bottlenecks in Pipelines:

Ideal Situation:

- Each stage **immediately** gets a new packet to process — no idling!
- All stages have **equal processing times** → **Load balanced** pipeline.

Real-World Problem:

- Some stages naturally become **bottlenecks** (slower than others).

Solution 1: Parallelization

- Split the bottleneck stage into **multiple concurrent stages**.
- Data is distributed across them to **reduce latency**.
- Example: Split one slow stage into two faster parallel stages.

Note:

- Parallelization introduces some **overhead** (extra management effort).

Solution 2: I/O Buffers

- Place **input/output buffers** between stages.
 - Buffers **hold data temporarily** to:
 - Smooth out differences in processing speeds.
 - Prevent stalls or overloads.
 - Buffers are crucial especially for **dynamic (nonlinear)** pipelines.
-

Key Takeaways:

- Pipelines involve more than ETL: they require **scheduling, monitoring, maintenance, and optimization**.
 - **Monitoring** tracks latency, throughput, resource usage, and failures.
 - **Parallelization** and **I/O buffers** help **mitigate bottlenecks** and keep data flowing smoothly.
-

Batch vs Streaming Data Pipelines

Objectives:

- Differentiate between **batch** and **streaming** data pipelines.
 - Understand **micro-batch** and **Lambda architecture**.
 - List **use cases** for both batch and streaming pipelines.
-

Batch Data Pipelines:

- **Definition:**
 - Operate on **large datasets** all at once.
- **Schedule:**
 - Periodic (every few hours, days, weeks) or triggered by conditions (e.g., file size).
- **When to use:**
 - **Accuracy is critical.**
 - **Recency of data is not important.**
- **Advantages:**
 - High **data quality** (e.g., cleaning, full validation).
- **Examples of Batch Use Cases:**
 - Periodic **data backups**.
 - **Processing customer orders** and billing.
 - **Data modeling** on slowly changing data.
 - **Sales and weather forecasting** (mid-to-long term).

- **Historical data analysis.**
 - **Diagnostic medical image processing.**
-

Streaming Data Pipelines:

- **Definition:**
 - Process data **record by record** as it **arrives** (real-time).
 - **When to use:**
 - **Minimal latency** is required.
 - Applications need **real-time insights**.
 - **Advantages:**
 - Immediate reaction to **events** and **new data**.
 - **Examples of Streaming Use Cases:**
 - **Watching movies**, music, or podcasts.
 - **Social media feeds** and **sentiment analysis**.
 - **Fraud detection** (e.g., credit card transactions).
 - **User behavior analysis**.
 - **Targeted advertising**.
 - **Stock market trading**.
 - **Real-time product pricing**.
 - **Recommender systems**.
-

Micro-Batch Processing:

- **Definition:**
 - **Small batches** processed **very frequently**.
 - **Why use it:**
 - Achieve **near real-time** processing.
 - Helps in **load balancing** and **reducing latency**.
 - Useful for **very short time windows** of data.
-

Lambda Architecture (Hybrid Approach):

- **Definition:**
 - Combines **batch processing** + **stream processing**.
- **Structure:**
 - **Batch Layer:** Historical data processed in bulk.
 - **Speed Layer:** Real-time data processed immediately.
 - **Serving Layer:** Combines both results for queries.
- **When to use:**
 - You need both **historical accuracy** and **real-time speed**.
- **Challenges:**

- **Complex design** and maintenance.
-

Key Takeaways:

- **Batch Pipelines:**
 - Best for **accuracy** and **large-scale, periodic processing**.
 - **Streaming Pipelines:**
 - Best for **low latency, real-time applications**.
 - **Micro-batch:**
 - Bridges the gap toward **real-time** without full streaming complexity.
 - **Lambda Architecture:**
 - Ideal when needing **both speed and historical completeness**, at the cost of **higher complexity**.
-

Data Pipeline Tools and Technologies

Learning Objectives

- Discuss key data pipeline technologies.
 - List open-source and enterprise ETL/ELT tools.
 - List streaming data pipeline tools.
-

Modern ETL/ELT Tool Features

- Fully automated pipeline creation (extraction → transformation → loading).
 - No-code ETL (drag-and-drop GUI for defining flows).
 - Support for complex transformations (string operations, calculations, data merging).
 - Built-in **security** (encryption in transit and at rest) and **compliance** (HIPAA, GDPR).
-

Python-Based Libraries

- **Pandas:** Great for prototyping ETL pipelines and exploratory data analysis (EDA). Handles data via *DataFrames* but has memory limitations.
- **Dask, Vaex, Apache Spark:** Scalable alternatives to Pandas for large datasets.
- **PostgreSQL:** SQL-based scalable alternative to DataFrame APIs.



Open Source Data Pipeline Tools

- **Apache Airflow**
 - Created by Airbnb.
 - Programmatically author, schedule, monitor workflows.
 - Highly scalable and cloud-integrated.
- **Talend Open Studio**
 - Supports big data migration, data profiling, and warehousing.
 - Drag-and-drop GUI (auto-generates Java code).
 - Connects to databases like Google Sheets, RDBMS, IBM DB2, Oracle.



Enterprise Data Pipeline Tools

- **AWS Glue**
 - Fully managed ETL.
 - Auto-discovers data schema via crawling.
 - Easy job creation via AWS Console.
- **Panoply**
 - Focuses on ELT pipelines.
 - No-code integration with SQL querying capabilities.
 - Works with BI tools like Tableau and Power BI.
- **Alteryx**
 - Self-service data analytics platform.
 - Drag-and-drop ETL creation without SQL/programming.
- **IBM InfoSphere DataStage**
 - ETL/ELT integration tool.
 - Drag-and-drop interface.
 - Supports parallel processing for scalability.



Streaming Data Pipeline Technologies

- **IBM Streams**
 - Real-time analytics platform (supports Java, Python, C++).
 - Analyzes millions of events/sec with sub-millisecond latency.
 - Includes drag-and-drop building via IBM Stream Flows.
- Other Tools:
 - **Apache Kafka**
 - **Apache Storm**
 - **Apache Samza**
 - **Apache Spark (Streaming)**
 - **SQL Stream**
 - **Azure Stream Analytics**



Key Takeaways

- Modern ETL/ELT tools feature automation, ease-of-use, and strict security compliance.
- **Pandas**, **Vaex**, **Dask** are excellent for prototyping but scalability may require **Spark** or **PostgreSQL**.
- **Apache Airflow** and **Talend Open Studio** help automate and manage big data workflows.
- **Panoply** focuses on ELT, while **Alteryx** and **InfoSphere DataStage** handle both ETL and ELT.
- **Streaming tools** like **Kafka**, **IBM Streams**, and **Spark** enable real-time processing with low latency.

Lesson' summary:

- Data pipelines move data from one place, or form, to another
- Data flows through pipelines as a series of data packets
- Latency and throughput are key design considerations for data pipelines
- Data pipeline processes include scheduling or triggering, monitoring, maintenance, and optimization
- Parallelization and I/O buffers can help mitigate bottlenecks
- Batch pipelines extract and operate on batches of data
- Batch processing applies when accuracy is critical, or the most recent data isn't required
- Streaming data pipelines ingest data packets one-by-one in rapid succession
- Streaming pipelines apply when the most current data is needed
- Examples of streaming data pipelines use cases, such as social media feeds, fraud detection, and real-time product pricing
- Modern data pipeline technologies include schema and transformation support, drag-and-drop GUIs, and security features
- Stream-processing technologies include Apache Kafka, IBM Streams, and SQLStream

Can you differentiate between ETL and ELT?

Drag and drop the below terms into the correct bins

Batch Processing		Stream Processing
Good for activities like payroll processing	Submit	Good for activities like weather sensor data processing.
Usually a scheduled activity		Realtime activity
MapReduce	Score 100 %	Spark Streaming
High Latency	Start Over	Low latency

Module Three

Building Data Pipelines using Airflow

Apache Airflow Overview

Learning Objectives

After this video, you'll be able to:

- Recognize Apache Airflow as a platform to programmatically author, schedule, and monitor workflows.
- List key features and principles of Apache Airflow.
- Identify common use cases.

What is Apache Airflow?

- An **open-source workflow orchestration tool**.
 - Used to build and run **batch data pipelines**.
 - Workflows are represented as **Directed Acyclic Graphs (DAGs)**—a set of tasks in a defined execution order.
 - It is **not** a data streaming or event-driven tool.
-

Core Components

Component	Description
Scheduler	Schedules DAG runs and determines when tasks should be executed.
Executor	Executes the tasks by assigning them to workers.
Workers	Perform the actual execution of the tasks.
Web Server (UI)	Provides a user interface to view, trigger, and debug DAGs and tasks.
Metadata DB	Stores the state and history of all DAGs and tasks.
DAG Directory	Location where DAG files are stored and accessed.

Task Lifecycle States

- **No status** – Task not yet queued.
 - **Scheduled** – Task is scheduled to run.
 - **Queued** – Task is waiting for an available worker.
 - **Running** – Task is being executed.
 - **Success** – Task completed successfully.
 - **Failed** – Task failed due to an error.
 - **Upstream Failed** – A dependent task failed.
 - **Up for retry** – Task is set to retry based on configuration.
 - **Removed** – Task was deleted after the DAG run started.
-

Key Features of Apache Airflow

1. **Pure Python** – Define workflows using standard Python code.
 2. **User Interface** – Monitor, schedule, and debug tasks via a powerful web UI.
 3. **Integrations** – Easily connects with external systems (e.g., IBM Databand).
 4. **Ease of Use** – Simple to create and deploy workflows with Python knowledge.
 5. **Open Source** – Community-supported with shared contributions and plugins.
-

Core Principles

1. **Scalable** – Modular and distributed architecture with message queue support.
 2. **Dynamic** – Pipelines are Python-based and allow dynamic task generation.
 3. **Extensible** – Custom operators and libraries can be defined easily.
 4. **Lean** – Explicit, parameterized design using Jinja templating.
-

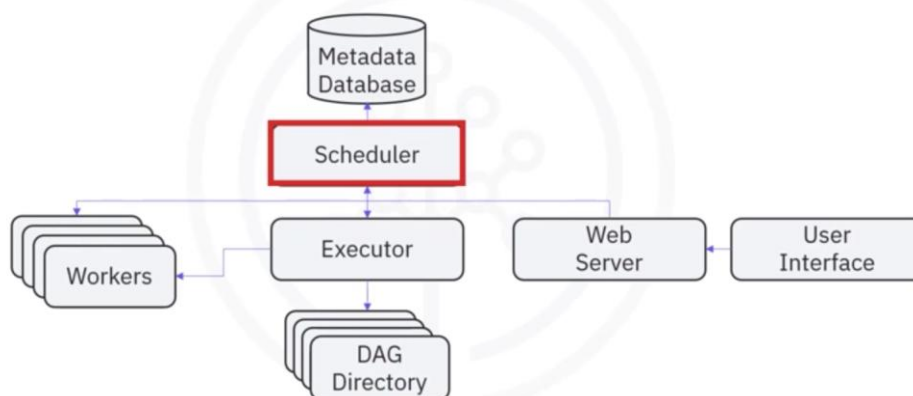
Common Use Cases

- **Adobe** – Custom operators for business-specific tasks.
 - **Adyen** – Simplified ETL development with reusable components.
 - **Big Fish** – Visual monitoring and programmatic control of workflows.
 - **Walmart** – Automating data extraction and loading into data warehouses.
 - **Machine Learning Pipelines** – Managing complex task dependencies.
-

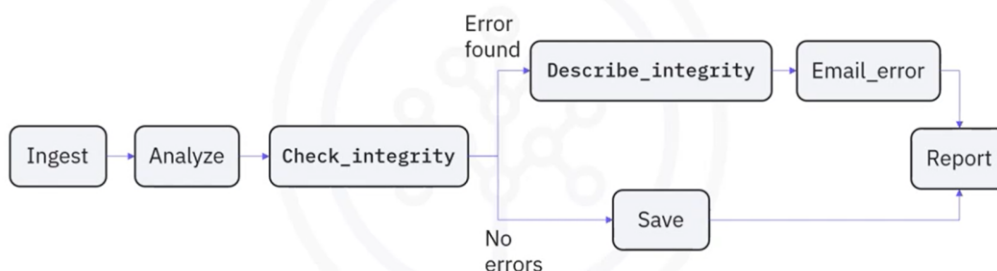
Recap / Key Takeaways

- Apache Airflow helps you **build, schedule, and monitor data workflows** using Python.
- It uses **DAGs** to define task dependencies and order of execution.
- Comes with a **scheduler, executor, web UI, workers, and metadata DB** to orchestrate pipelines.
- Key strengths include **flexibility, integration support, ease of use, and extensibility**.
- It's ideal for **ETL processes, batch data pipelines, and machine learning workflows**.

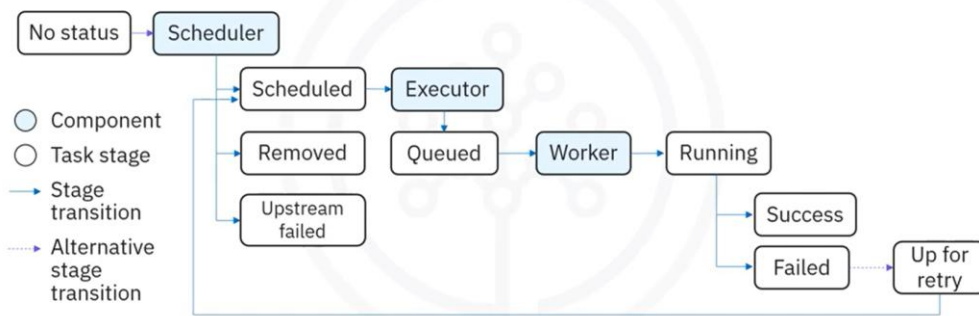
Simplified view of Airflow's architecture:



Sample DAG illustrating the labeling of different branches:



The lifecycle of an Apache Airflow task state:



Advantages of Representing Data Pipelines as DAGs in Apache Airflow

Learning Objectives

After watching this video, you will be able to:

- Define what a **DAG** is.
- Describe workflows as **DAGs of tasks and dependencies**.
- Outline the **components of a DAG definition file**.
- Explain how the **Airflow Scheduler executes tasks** across workers.
- List the **benefits of defining workflows as code**.

What is a DAG?

- **DAG** stands for **Directed Acyclic Graph**.
- It's a graph made of **nodes (tasks)** and **directed edges (dependencies)**.
- "Acyclic" means there are **no cycles or loops**—a task can't depend on itself.
- Common DAG examples include trees and task workflows.

DAGs in Apache Airflow

- DAGs model your **workflow or data pipeline**.
 - Each **task** is a node.
 - Each **dependency** between tasks is a directed edge, indicating execution order.
 - DAGs are defined in **Python scripts** and treated as **code**.
-

Components of a DAG File

1. **Library Imports** – Import necessary modules (e.g., `from airflow import DAG`).
 2. **DAG Arguments** – Default settings like `start_date`, `retries`, etc.
 3. **DAG Definition** – Instantiates the DAG using parameters and arguments.
 4. **Task Definitions** – Individual tasks using operators (e.g., `BashOperator`, `PythonOperator`).
 5. **Task Pipeline** – Defines the task dependencies (e.g., `task1 >> task2`).
-

Operators and Sensors

- **Operators** define what a task does (e.g., `BashOperator`, `PythonOperator`, `SQLOperator`).
 - **Sensors** wait for a condition (e.g., file exists, DAG completes).
 - Additional operators exist for emails, HTTP requests, etc.
-

Airflow Scheduler & Task Execution

- **Scheduler** is a background service that reads DAGs and triggers tasks.
 - It **distributes tasks** across multiple **workers** based on dependencies and schedules.
 - DAGs run based on the **start date** and **schedule interval** specified in the DAG code.
-

Advantages of Defining Workflows as Code

- **Maintainability** – Easy to understand and update.
 - **Version Control** – Track changes using Git or similar tools.
 - **Collaboration** – Multiple developers can work on workflows together.
 - **Testability** – Code can be validated with unit tests.
 - **Clarity** – Execution logic is explicitly defined in code.
-

Recap / Key Takeaways

- A **DAG** is a **Directed Acyclic Graph**—ideal for modeling data workflows.
- Apache Airflow uses DAGs to **organize and schedule tasks** with clear dependencies.
- DAGs are **defined as Python code**, making them:
 - **Readable**

- **Reusable**
 - **Collaborative**
 - **Version-controlled**
 - **Testable**
 - The **Scheduler** automatically runs DAGs and assigns tasks to **available workers**.
 - Airflow promotes **modular, maintainable**, and **scalable** workflow design.
-

Apache Airflow UI

Learning Objectives

After watching this video, you will be able to:

- Identify existing **DAGs** in your Airflow environment.
 - List different ways to **visualize a DAG**.
 - Review the **Python code** that defines a DAG.
 - Analyze **task durations** over multiple DAG runs.
 - Access **context metadata** for any task instance.
-

Airflow UI Overview

- The **landing page** shows the **DAGs view** – a table listing all DAGs.
 - Each row includes:
 - **DAG name**
 - **Owner** (e.g., "airflow" for built-in DAGs)
 - **Current task status**
 - **Run schedule** (in crontab format)
 - **History of past runs**
 - **Last/next run time**
 - **Quick access links**
 - You can **pause/resume** DAGs using the toggle button.
-

Visualizing DAGs

1. **Grid View**
 - Timeline of task statuses per DAG run.
 - Color-coded status indicators.
 - Hover to view task info.
 - Choose number of runs and base date.
2. **Graph View**
 - Visual graph of tasks and dependencies.
 - Tasks color-coded by operator type.

- Example: `print_hello` (BashOperator) → `execute_function` (PythonOperator).
 - Filter by task status.
-

Inspecting Tasks and Code

- **Task Instance Context Menu** (accessed via grid/graph):
 - View/edit task details.
 - Access **log files** for the task.
 - **Code View**:
 - Full Python script defining the DAG.
 - Shows **imports**, **default arguments**, and **task/operator definitions**.
 - **Task Duration View**:
 - Timeline chart of task performance.
 - Toggle to highlight specific runs.
-

Recap / Key Takeaways

- The **Airflow UI** provides a rich, interactive dashboard for managing workflows.
 - You can view DAGs in **grid**, **graph**, and **code** modes for better understanding.
 - **Task context menus** allow deep inspection and debugging.
 - The **Code tab** reveals the exact Python script defining the DAG.
 - The **Task Duration** tab helps track performance trends over time.
 - Overall, the UI enhances **observability**, **debugging**, and **transparency** in managing DAGs.
-

Build a DAG Using Airflow

Learning Objectives

After this video, you will be able to:

- Understand that an Airflow pipeline is a **Python script** defining a **DAG (Directed Acyclic Graph)**.
 - List the **key components** of a DAG definition file.
 - **Create tasks** using operators inside the DAG script.
 - Define **dependencies** between tasks.
-

Key Components of a DAG Definition

An Apache Airflow DAG file (e.g., `simple_example_DAG.py`) includes the following logical blocks:

1. Library Imports

```
from airflow.models import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta
```

2. Default Arguments

```
default_args = {
    'owner': 'you',
    'start_date': datetime(2024, 1, 1),
    'retries': 1,
    'retry_delay': timedelta(minutes=5)
}
```

3. DAG Instantiation

```
dag = DAG(
    dag_id='simple_example',
    description='A simple example DAG',
    default_args=default_args,
    schedule_interval='*/5 * * * *' # Every 5 seconds (not ideal
in real usage)
)
```

4. Task Definitions

```
task1 = BashOperator(
    task_id='print_hello',
    bash_command='echo "Greetings, the date and time are"',
    dag=dag
)

task2 = BashOperator(
    task_id='print_date',
    bash_command='date',
    dag=dag
)
```

5. Task Pipeline / Dependencies

```
task1 >> task2 # task1 runs first, then task2
```

Recap / Key Takeaways

- An **Airflow DAG** is just a **Python script**.
- A DAG file includes:
 - **Imports**

- **Default arguments**
 - **DAG object instantiation**
 - **Task creation** using operators (e.g., BashOperator)
 - **Dependencies** between tasks using `>>` or `<<`
- You can **schedule runs** using the `schedule_interval` parameter (e.g., crontab).
- **Tasks = operators** bound to the DAG object.

Airflow Logging and Monitoring

Learning Objectives

After this video, you will be able to:

- Use **logging features** to monitor task status and **diagnose problems** in DAG runs.
- Explain how to **access and understand emitted metrics** like counters, gauges, and timers.

Logging in Airflow

◆ Purpose

- Logs help **monitor tasks** and **debug issues** during DAG execution.

◆ Log Storage Options

- **Default:** Local file system (development environments).
- **Production:**
 - Cloud storage (e.g., **IBM Cloud, AWS, Azure**)
 - Log analyzers (e.g., **Elasticsearch, Splunk**) for indexing and search.

◆ Log File Structure

Logs are stored using a directory path pattern:

```
logs/dag_id=<dag_id>/run_id=<run_id>/task_id=<task_id>/attempt=<attempt>.log
```

Example:

```
logs/dag_id=dummy_dag/run_id=scheduled_2024-01-01T00:00:00Z/task_id=task1/attempt=1.log
```

◆ Log Content

- Running command
 - Command result
 - Task result
 - Task event logs
-

Airflow UI for Logs

- The **Airflow Web UI** allows:
 - Searching task logs by `dag_id`, `task_id`, and `logical_date`
 - Quickly reviewing events for task instances
-

Metrics in Airflow

Airflow emits three types of runtime metrics:

Type	Description	Example
Counter	Always increases	Total number of failed or successful tasks
Gauge	Fluctuates (snapshot metrics)	Number of currently running tasks
Timer	Measures time duration	Time taken for a task to complete

Recommended Tools

- **StatsD**: Collects metrics and forwards them
 - **Prometheus**: Stores, aggregates, and visualizes metrics in dashboards
-

Key Takeaways

- Logs can be stored locally or pushed to **cloud/search engines** (Elasticsearch, Splunk).
 - **Log files** follow a specific structure and can be accessed via file system or Airflow UI.
 - Metrics are categorized into **Counters**, **Gauges**, and **Timers**.
 - Use **StatsD** for metric emission and **Prometheus** for visualization and monitoring.
-

Lesson's Summary:

- Apache Airflow is scalable, dynamic, extensible, and lean
 - The five main features of Apache Airflow are pure Python, useful UI, integration, easy to use, and open-source
 - A common use case is that Apache Airflow defines and organizes machine learning pipeline dependencies
 - Tasks are created with Airflow operators
 - Pipelines are specified as dependencies between tasks
 - Pipeline DAGs defined as code are more maintainable, testable, and collaborative
 - Apache Airflow has a rich UI that simplifies working with data pipelines
 - You can visualize your DAG in graph or grid mode
 - Key components of a DAG definition file include DAG arguments, DAG and task definitions, and the task pipeline
 - Set a schedule to specify how often to re-run your DAG
 - You can save Airflow logs into local file systems and send them to cloud storage, search engines, and log analyzers
 - Airflow recommends sending production deployment logs to be analyzed by Elasticsearch or Splunk
 - You can view DAGs and task events with Airflow's UI
 - The three types of Airflow metrics are counters, gauges, and timers
 - Airflow recommends that production deployment metrics be sent to and analyzed by Prometheus via StatsD
-

Module Four

Building Streaming Pipelines using Kafka

Distributed Event Streaming Platform Components

Learning Objectives

After watching this video, you will be able to:

- Define what an **event** is in event streaming.
 - Identify **common event formats**.
 - Explain what an **Event Streaming Platform (ESP)** is.
 - List the **main components** of an ESP.
 - Recognize **popular ESP tools** like Kafka, Flink, and others.
-

What is an Event?

An **event** is a data record representing an entity's **state change over time**.

Examples:

- GPS coordinates of a car.
 - Temperature readings in a room.
 - Blood pressure of a patient.
 - RAM usage of a running application.
-

Common Event Formats

Format Type	Example
Primitive	Plain text, numbers, dates
Key-Value	{ "car_id": 1, "location": [lat, long] }
Key-Value + Timestamp	{ "patient_id": "pt001", "bp": [120, 80], "timestamp": "..."} }

What is Event Streaming?

Event streaming is the **continuous transportation of events** from sources to destinations.

Components Involved:

- **Event Sources:** Sensors, apps, databases, monitoring systems.
- **Event Destinations:** File systems, apps, other databases.

! Real-World Challenge:

Handling **many-to-many** relationships between sources and destinations with different protocols (FTP, HTTP, JDBC, SCP) becomes complex.

✿ What is an Event Streaming Platform (ESP)?

An **ESP** is a **middleware system** that handles and simplifies event transport by acting as a centralized event router.

🔄 Simplified Flow:

- **Sources send** events to the ESP.
 - **Destinations subscribe** to ESP to **receive events**.
-

🧱 Main Components of an ESP

1. **Event Broker (Core Component)**
 - Manages receiving and delivering events.
 - Subcomponents:
 - **Ingestor:** Receives events from sources.
 - **Processor:** Transforms events (serialization, compression, encryption).
 - **Consumption:** Distributes events to subscribers.
 2. **Event Storage**
 - Temporarily holds events so destinations can fetch them at any time.
 3. **Analytic & Query Engine**
 - Enables querying and analyzing stored event data.
-

🌐 Popular Event Streaming Platforms

Platform	Description
Apache Kafka	Most popular open-source ESP
Amazon Kinesis	AWS-managed event streaming
Apache Flink	Event processing and real-time analytics
IBM Event Stream	Enterprise-grade ESP from IBM
Azure Event Hub	Microsoft Azure's ESP

✓ Key Takeaways

- An **event** is an observable state update.
 - Events come in **primitive**, **key-value**, or **timestamped** formats.
 - **ESP** simplifies the complex routing between sources and destinations.
 - Core ESP components include **event broker**, **storage**, and **query engine**.
 - **Kafka** is the most widely used ESP; others include **Flink**, **Kinesis**, **IBM Event Stream**, and **Azure Event Hub**.
-

Apache Kafka overview

Objectives:

- Identify Apache Kafka as an Event Streaming Platform (ESP)
 - Describe Kafka's architecture
 - List common Kafka use cases
 - Summarize its key features and benefits
 - Name popular Kafka-based ESP service providers
-

What is Apache Kafka?

Apache Kafka is an **open-source, distributed event streaming platform** used to:

- Ingest, store, process, and distribute real-time event data
 - Serve as middleware between producers (data sources) and consumers (data sinks)
-

Common Use Cases:

- **User behavior tracking:** clicks, page views, gestures
 - **IoT & telemetry:** sensor readings, GPS
 - **Log aggregation:** system and app logs
 - **Finance:** payment processing, transactions
 - **Real-time analytics:** dashboards, AI/ML pipelines
 - **Notifications:** emails, SMS, alerts
 - **Data compliance:** auditing sensitive data
-

Kafka Architecture

Component	Description
Brokers	Servers that receive, store, and distribute events
Producers	Clients that publish data to topics
Consumers	Clients that subscribe to and consume data from topics
Topics	Logical channels that group event streams
Kafka Connect	Imports/exports data between Kafka and external systems
Controllers	Coordinate metadata and cluster state

Kafka now uses **KRaft (Kafka Raft)** for metadata management (replaces Zookeeper from version 2.8+)

Key Features of Kafka

-  **High throughput & scalability**
-  **Durable storage with partitions & replication**
-  **Fault-tolerant & reliable**
-  **Replayable events** (no deadline to consume)
-  **Open-source & extensible**
-  **Supports multiple client APIs** (Java, Python, Go, REST, etc.)

Kafka-Based ESP Providers

Provider	Key Offering
Confluent Cloud	Fully managed Kafka (on-prem & cloud)
IBM Event Streams	Kafka with enterprise features like security & monitoring
Amazon MSK (Managed Streaming for Kafka)	Fully managed Kafka on AWS

Final Notes

- Kafka is ideal for **reliable, scalable, real-time data pipelines**
 - Despite being open-source, deploying Kafka at scale requires **significant expertise**
 - Managed services reduce operational complexity for enterprises
-

Building Event Streaming Pipelines using Kafka

Objectives

After watching this video, you should be able to:

- Describe Kafka's core components
 - Publish and subscribe to streams of events using Kafka
 - Consume events live or replay past events
 - Understand a full end-to-end Kafka event streaming pipeline
-

Kafka Core Components

Component	Description
Brokers	Dedicated servers that store, process, and distribute events
Topics	Logical containers for events, like tables in a database
Partitions	Subdivisions of topics across brokers to enable parallel processing
Replications	Copies of partitions for fault tolerance
Producers	Client apps that publish events to topics
Consumers	Client apps that read events from topics

Kafka CLI Tools

- **kafka-topics**: Manage topics (create, list, describe, delete)
 - **kafka-console-producer**: Publish events to a topic
 - **kafka-console-consumer**: Subscribe to and read events from a topic
-

Publishing Events

- Producers can send events **with or without keys**
 - Events with the same key go to the same partition
 - Example:
 - Publish logs from Event Source 1 to log_topic
 - Publish user activity from Event Source 2 to user_topic
-



Consuming Events

- Consumers subscribe to topics and read events **in the order** they were published
 - Kafka tracks **offsets** (position of the last read event)
 - You can:
 - Read from the **latest offset**
 - Rewind and read **from the beginning** (offset = 0)
-



End-to-End Pipeline Example

Use case: Correlate **weather** data and **Twitter** posts about extreme weather.

1. **Event Sources:**
 - IBM Weather API → JSON
 - Twitter API → JSON
 2. **Kafka Topics:**
 - `weather_topic`
 - `twitter_topic`
 3. **Producers:**
 - Weather Producer → Kafka
 - Twitter Producer → Kafka
 4. **Consumers:**
 - Read and deserialize events
 5. **DB Writer:**
 - Parse JSON → Create DB records
 6. **Database + Dashboard:**
 - Store and visualize/analyze event data
-



Final Takeaways

- Kafka allows **scalable, reliable** real-time and replayable data pipelines
 - Producers and consumers are **fully decoupled**
 - Events are stored **durably** in Kafka topics and can be processed anytime
 - The core components of Kafka are brokers, the dedicated servers to receive, store, process, and distribute events.
-

Kafka Streaming Process

Lesson Objectives

- Understand what the **Kafka Streams API** is.
 - Recognize its **main benefits**.
 - Learn about the **Stream Processing Topology** concept.
-

Introduction: The Need for Real-Time Data Processing

In **event streaming**, data engineers don't just transmit data—they also need to **process** it:

- Filtering
- Aggregation
- Enrichment

Applications built for this purpose are called **stream processing applications**.

Traditional Approach: Ad Hoc Processing

Example:

1. Request raw weather data from a weather API.
2. A **producer** sends this data to `raw_weather_topic`.
3. A **consumer** reads from this topic.
4. An **ad hoc processor** (script or custom app) filters the data (e.g., detects extreme weather).
5. The filtered data is published to `processed_weather_topic`.
6. A final **consumer** reads it and sends it to a dashboard.

 **Problem:** As the number of topics grows, the system becomes complex and harder to maintain.

Solution: Kafka Streams API

What is it?

- A **lightweight client library** (Java-based) for processing data directly in Kafka.
- Both input and output are Kafka **topics**.
- Processes **one record at a time**.
- Ensures **exactly-once processing** for reliability.

Stream Processing Topology

A **computational graph** representing the data processing pipeline.

Components:

Processor Type	Description
Source Processor	Reads from Kafka (acts like a consumer).
Stream Processor	Transforms data (e.g., filters, aggregates).
Sink Processor	Writes to Kafka (acts like a producer).

Edges in the graph represent **streams**, and nodes represent **processors**.

Refactoring the Example Using Kafka Streams

Instead of multiple custom components:

- **Source Processor:** Reads from `raw_weather_topic`.
- **Stream Processor:** Filters high-temperature events.
- **Sink Processor:** Publishes results to `processed_weather_topic`.

✅ **Result:** A much simpler, scalable, and maintainable architecture.

✅ Key Takeaways

- **Kafka Streams API** is a powerful tool for stream processing **within** Kafka.
 - It simplifies building data pipelines.
 - Uses a clear **topology structure** of source, processing, and sink.
 - Designed for **scalability**, **reliability**, and **ease of use**.
-

Kafka Python Client

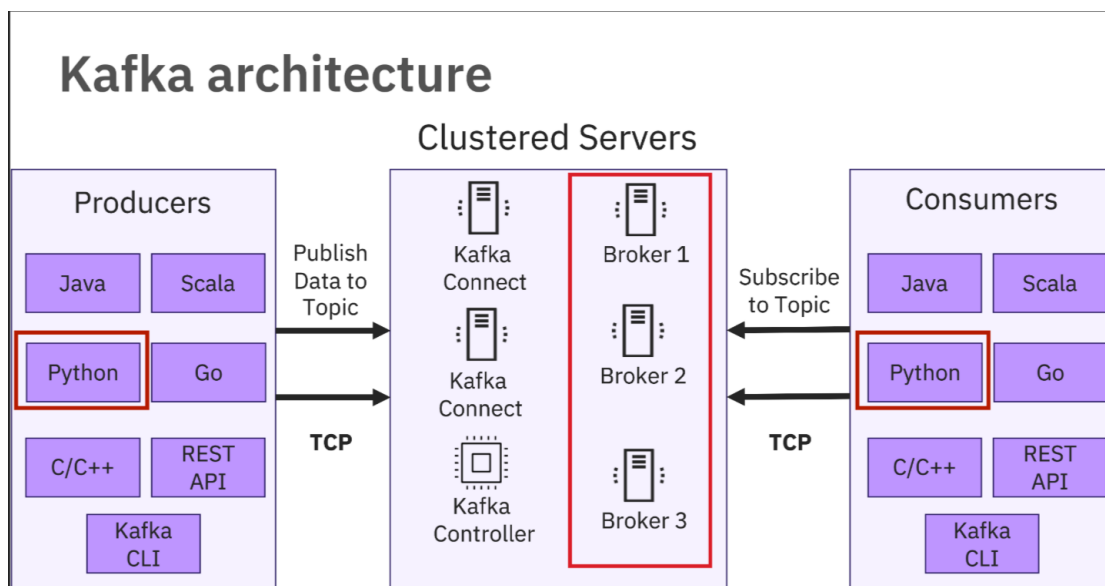
Objectives

After completing this lesson, you will be able to:

- List common Apache Kafka clients.
- Use `kafka-python` to interact with a Kafka server in Python.

Kafka Architecture Overview

- **Kafka Server (Broker):** A cluster of brokers that receive, store, and distribute events.
- **Kafka Connect:** Moves data in and out of Kafka as streams.
- **ZooKeeper (Pre-2.8):** Used for cluster coordination.
- **KRaft (Post-2.8):** Replaces ZooKeeper with built-in Kafka Controllers for metadata management.



Kafka Clients

Kafka provides multiple client interfaces:

Client Type	Description
Kafka CLI	Shell scripts for managing Kafka (e.g., topics, messages).
High-Level APIs	Clients in Python, Java, Scala, etc.
REST API	Web API for Kafka operations.
Community Clients	Third-party Kafka clients maintained by the community.

kafka-python Client

kafka-python is a Python client library for Apache Kafka.

Installation

```
pip3 install kafka-python
```

✂ KafkaAdminClient

Create Admin Client

```
from kafka.admin import KafkaAdminClient, NewTopic
admin_client = KafkaAdminClient(bootstrap_servers="localhost:9092",
client_id='test')
```

Create a Topic

```
topic_list = []
new_topic = NewTopic(name="bankbranch", num_partitions=2,
replication_factor=1)
topic_list.append(new_topic)
admin_client.create_topics(new_topics=topic_list)
```

Describe a Topic

```
from kafka.admin import ConfigResource, ConfigResourceType
configs = admin_client.describe_configs(
    config_resources=[ConfigResource(ConfigResourceType.TOPIC,
"bankbranch")])
```

✉ KafkaProducer

Create Producer

```
from kafka import KafkaProducer
import json

producer = KafkaProducer(value_serializer=lambda v:
json.dumps(v).encode('utf-8'))
```

Send JSON Messages

```
producer.send("bankbranch", {'atmid': 1, 'transid': 100})
producer.send("bankbranch", {'atmid': 2, 'transid': 101})
```

KafkaConsumer

Create Consumer

```
from kafka import KafkaConsumer
```

```
consumer = KafkaConsumer('bankbranch')
```

Consume Messages

```
for msg in consumer:  
    print(msg.value.decode("utf-8"))
```



Summary

- `kafka-python` allows you to create and manage Kafka topics, produce messages, and consume them—all in Python.
 - It mirrors CLI operations like `kafka-topics.sh`, `kafka-console-producer.sh`, and `kafka-console-consumer.sh`.
 - JSON is commonly used for message content, requiring serialization to bytes for Kafka.
-

Summary & Highlights

- An event stream represents entities' status updates over time
 - The main components of an ESP are Event broker, Event storage, Analytic, and Query Engine
 - Apache Kafka is a very popular open-source ESP
 - Popular Kafka service providers include Confluent Cloud, IBM Event Stream, and Amazon MSK
 - The core components of Kafka are brokers, topics, partitions, replications, producers, and consumers
 - The `Kafka-console-consumer` manages consumers
 - Kafka Streams API is a simple client library supporting you with data processing in event streaming pipelines
 - A stream processor receives, transforms, and forwards the processed stream
 - Kafka Streams API uses a computational graph
 - There are two special types of processors in the topology: The source processor and the sink processor
-